Reliable Data Transfer over UDP — Project Report

Course: CS-30003 — Coding Assignment 1

Project: P3 — Reliable Data Transfer over UDP

Team: Anuj Sharma, Bhuvika Naulani, Divyanshi, Dhruv

Section: CSE 33

1. Overview

This project implements a file-transfer application over UDP with three selectable ARQ protocols: Stop-and-Wait, Go-Back-N, and Selective Repeat. The implementation adds sequence numbers, a packet checksum, acknowledgements, retransmission timers, adaptive RTO logic, a configurable channel emulator, and whole-file SHA-256 verification. The required experiments compare protocol goodput, window size, reordering and RTO settings.

2. Design summary

• **Transport:** real UDP sockets; one sender/server and one receiver/client per test run.

• **Packet format:** 10-byte header followed by 0–1024 bytes of payload. Header fields are type (1 byte), reserved (1 byte), sequence/ACK number (4 bytes), payload length (2 bytes), and Internet checksum (2 bytes). A zero-length DATA packet marks EOF.

• **Integrity:** Internet checksum detects packet corruption; SHA-256 comparison verifies the complete received file. These checks serve different purposes.

• **ARQ:** Stop-and-Wait retransmits one outstanding packet; GBN uses a sliding window and cumulative ACKs; SR uses per-packet ACK/timer state and buffers in-window out-of-order data.

• **RTO:** Jacobson/Karels SRTT/RTTVAR estimation, Karn sampling rules, timeout backoff, and a fixed-RTO mode for the sensitivity experiment.

• **Channel emulator:** configurable loss, duplication, reordering, corruption, delay and jitter. Experiment runs record a base seed and separate deterministic DATA/ACK seeds.

3. Verification results

Fresh full-suite run on the 2026-10-24 source revision: make test exited with code 0. The full output is retained in verification/fresh_make_test_2026-10-24.log and the exit code in the matching .exit file.

Suite	Result	Notes
Clean build (-Wall -Wextra)	PASS	No visible compiler warnings.
Unit tests	26/26 PASS	Packet validation, checksum, RTO and SHA-256 tests.
Channel emulator tests	PASS	Includes timing/determinism checks and immediate invalid-socket error reporting.
End-to-end tests	93/93 PASS	All three protocols, packet loss/corruption, delay, EOF and retry-cap cases.
GBN packet-level tests	24/24 PASS	Window, cumulative ACK, retransmission and EOF cases.
SR packet-level tests	27/27 PASS	Selective ACK, buffering, per-packet timer and retransmission cases.
Hash-integrity suite	48/48 PASS	File-size coverage, impairment cases, wrong-hash detection and byte comparison.

The packaged A–D experiment CSVs contain 180 historical rows and pass a numeric/data-field audit. Their original per-run server/client logs are not packaged, so those historical SHA-256 lines cannot be independently rechecked from this ZIP. Fresh one-repetition runs of A–D passed on the current source: 60/60 transfers, with the real server/client logs and SHA-256 values verified by verification/audit_fresh_experiments.py. These fresh runs are stored separately under verification/fresh_[A-D]_raw.csv and experiments/results/logs/fresh_[A-D]_raw/; they validate all configurations once but do not replace the three-repetition datasets used in the result tables below.

4. Experimental method

Each experiment uses a deterministic 1 MiB input file, 10 ms one-way delay per direction, no jitter unless otherwise specified, and three repetitions per configuration. The experiment runner applies the same impairment probabilities to both directions while using DATA seed = base seed and ACK seed = base seed + 1,000,003. Raw CSVs include status, exit codes, file/hash verification, transfer metrics and both directional seeds. Failed runs, if any, remain in the raw CSV and are excluded from means.

Goodput is the sender-reported file_bytes × 8 / time_ms, in kbps. Retransmissions are the sender-reported count. The experiment-specific reports include configuration tables and per-setting statistics.

5. Results overview

A. Goodput vs packet-loss probability

Protocol	Mean goodput at 0% loss (kbps)	Mean goodput at 15% loss (kbps)	Mean retransmissions at 15% loss
Stop-and-Wait	396.1	218.8	387.3
Go-Back-N	3161.4	737.6	1528.7
Selective Repeat	3169.9	764.8	387.0

Raw rows: 45; successful: 45; failed: 0. See experiment_A.md (experiments/results/experiment_A.md) and experiment_A_goodput_vs_loss.png (experiments/results/experiment_A_goodput_vs_loss.png).

B. Goodput vs window size

Protocol	Best measured mean-goodput window	Mean goodput (kbps)
Go-Back-N	32	6385.8
Selective Repeat	32	6417.7

Raw rows: 36; successful: 36; failed: 0. See experiment_B.md (experiments/results/experiment_B.md) and experiment_B_goodput_vs_window.png (experiments/results/experiment_B_goodput_vs_window.png).

C. Retransmissions vs reordering probability

Protocol	Mean retransmissions at 0% reordering	Mean retransmissions at 20% reordering
Stop-and-Wait	2.0	265.3
Go-Back-N	2.7	1591.0
Selective Repeat	2.7	48.7

Raw rows: 36; successful: 36; failed: 0. See experiment_C.md (experiments/results/experiment_C.md) and experiment_C_retransmissions_vs_reordering.png (experiments/results/experiment_C_retransmissions_vs_reordering.png).

D. RTO sensitivity

Protocol	Adaptive mean goodput (kbps)	Best fixed RTO by measured mean goodput	Mean goodput (kbps)
Stop-and-Wait	340.7	20 ms	363.2
Go-Back-N	1670.1	20 ms	2493.7
Selective Repeat	1781.9	20 ms	2310.2

Raw rows: 63; successful: 63; failed: 0. See experiment_D.md (experiments/results/experiment_D.md) and experiment_D_rto_sensitivity.png (experiments/results/experiment_D_rto_sensitivity.png).

6. Limitations and reproducibility notes

• Measurements use localhost with emulated delay and no bandwidth cap; they should not be treated as real-network throughput benchmarks.

• OS scheduling can trigger occasional spurious retransmissions when measured RTT is close to RTO. Three repetitions help expose variability but do not establish statistical significance.

• Fixed RTO is a comparison mode and is not the recommended operating mode.

• The latest source hardens packet validation, strict CLI parsing, file read/write/flush errors, channel queue allocation and pthread initialization, immediate UDP send failures, and queue overflow. Asynchronous send failures are counted in channel

stats because they occur after the originating call has returned; rare OS/resource-failure paths are not exhaustively verified.

• The packaged project intentionally excludes the old embedded .git directory because its history does not prove all four members' contributions. The team should create a fresh history in the empty GitHub repository and commit genuine work under each member's own Git identity.

• Review AI-USE.md and complete it with all AI use by all team members before submission.

7. Files and reproduction

• Historical raw data + metadata: experiments/results/experiment_[A-D]_raw.csv and matching .meta.txt files (three repetitions per configuration; original per-run logs are not packaged).

• Fresh one-repetition validation: verification/fresh_[A-D]_raw.csv, matching metadata, audit result and per-run logs under experiments/results/logs/fresh_[A-D]_raw/.

• Summaries and plots: experiments/results/experiment_[A-D]_summary.csv and the experiment PNGs.

• Written explanations: experiments/results/experiment_[A-D].md.

• Commands: README.md and experiments/README.md.

• One-page proposal: PROPOSAL.pdf (editable source: PROPOSAL.md).

• Design details and test notes: HANDOFF.md.

• AI disclosure draft: AI-USE.md (must be completed and confirmed by all team members).